Software Requirements and Backend Plan

Oct 7, 2026 · @Kevin

Frontend requirements

The app has seven views. All 4 main views must display a taskbar at the bottom.

Login

- Has a register button
- Has space for the user to enter a phone number
- Has a "Login" button that submits the phone number
- The phone number is sent to the backend and authenticated
- After authentication the user is logged in and taken to the device status page

Registration

- Two input fields: Name and Phone Number
- A register button below them
- One click checks that the inputs are valid, then calls a RegisterUser function
- RegisterUser sends the data to the backend, where the user is registered
- After registration the user is taken to the device status page

Bluetooth help

- A textbox with instructions on how to connect to the device
- A link to Apple's services for more in-depth Bluetooth help

Device status

- Shows which device the user is connected to, and its status
- Shows all previous and saved devices
- Users can choose another device to connect to
- Users can disconnect from a connected device
- Users can scan for devices to connect to

Data display / data history

- Shows a list of all data requests from the hardware

- The list is clickable; each item opens a detail view
- Shows an icon for whether the data has been uploaded

Data detail view

- An upload data button that sends the data to the backend
- Shows the time and location the data was pulled
- Shows charts and graphs of the information from the request

Account settings

- A box showing the user's name, phone number, collection preferences (meters, feet) and a notification on/off toggle
- A box showing the device currently connected and an auto-connect toggle
- A log out button that logs the user out and returns to the login page

Hardware

Open question: should users be able to upload multiple data sets at once, or one by one only? The backend plan below accepts a list on upload, so either answer works.

Backend stack

The backend is a Python async API on FastAPI, storing data in MongoDB.

Part	Choice
Framework	FastAPI (Python), run with uvicorn
Database	MongoDB
Database access	Motor (async driver) + Beanie (async ODM)
Validation	Pydantic
Dependencies and env	uv (or pip + venv if people prefer)
Testing	pytest + httpx
Lint and format	Ruff
Authentication	AWS Cognito user pool (sign-up, login, phone verification)

Part	Choice
Token checking	PyJWT, verifying Cognito tokens with the pool's public keys
Config	python-dotenv / pydantic-settings (Cognito region, user pool id, app client id)

API endpoints

Cognito handles sign-up and login, so the backend has no register or login endpoints. Every endpoint needs a valid Cognito token.

Endpoint	Used by	What it does
POST /me	Registration	Creates the user's profile right after Cognito sign-up
GET /me	Account settings	Returns name, phone, units, notifications, auto-connect
PATCH /me	Account settings	Updates units, notifications, auto-connect
GET /devices	Device status	Lists saved devices and the connected one
POST /devices	Device status	Saves a device after the user connects
PATCH /devices/{id}	Device status	Marks it connected or disconnected
DELETE /devices/{id}	Device status	Removes a saved device
GET /readings	Data history	Lists uploaded readings, newest first
GET /readings/{id}	Data detail view	Returns time, location and chart data
POST /readings	Data detail view	Uploads a list of readings (one or many)

Logout needs no endpoint: the app signs out of Cognito and returns to the login page.

Auth

Cognito owns sign-up, login and phone verification. The backend only checks tokens.

- The app signs up and logs in against the Cognito user pool and gets tokens back
- The app sends the token in the Authorization header on every request
- A FastAPI dependency checks the token (signature, expiry, issuer, app client) with PyJWT, then loads the user by the token's sub claim; a bad or expired token returns 401
- Right after sign-up the app calls POST /me, which creates the user's profile in MongoDB
- The user pool id, region and app client id live in pydantic-settings, loaded from a .env file
- Cognito can verify a phone number with a texted code, which closes the gap of logging in with a number alone; which sign-in flow the pool uses is still to decide

Open questions

- [ ] Upload one data set at a time, or many at once? (The backend accepts a list either way.)
- [ ] Data history lists all hardware requests, but the backend only stores uploaded ones. Should the app keep the not-yet-uploaded readings on the phone?
- [ ] Which Cognito sign-in flow: phone number with a texted code, or phone number plus a password?
- [ ] Cognito expects phone numbers in international format (for example +1 and digits). Should the app format them before sign-up?